

Secure End-to-End Messaging System

Design Document & Security Justification

Information Security – Spring 2026

1. System Architecture Overview

1.1 High-Level Design

The system consists of two clients (sender and receiver) communicating through a central server over HTTPS. The core architectural principle is that the server is **explicitly untrusted** for message confidentiality. It stores only ciphertext, enforces access control, and acts as a public key registry — but it can never read the contents of any message.

1.2 Component Roles

Component	Responsibilities	What it CANNOT do
Client (sender)	Generate RSA key pair Encrypt message with AES-GCM Wrap AES key with RSA-OAEP Send ciphertext bundle over TLS	See recipient's private key
Client (receiver)	Fetch encrypted messages Decrypt AES key with private RSA key Decrypt message with AES-GCM Display plaintext	Access other users' messages (403)
Server	Register users (store password hash) Issue session tokens Store public keys Store encrypted messages Enforce ACLs	Decrypt any message Recover passwords Read session content

2. Security Component Justifications

2.1 Why Hybrid Encryption is Used

Asymmetric encryption alone (RSA) cannot efficiently encrypt messages of arbitrary length. RSA-2048 can encrypt at most ~214 bytes of data, and it is computationally expensive — orders of magnitude slower than symmetric ciphers. Symmetric encryption alone (AES) is fast but introduces a key distribution problem: how does the sender securely share the AES key with the recipient without interception?

Hybrid encryption solves both problems simultaneously:

- **AES-256-GCM** encrypts the message itself. It is fast, hardware-accelerated, and handles arbitrary message lengths. GCM mode also provides authentication (integrity verification) via its authentication tag.
- **RSA-OAEP** encrypts only the small 32-byte AES key using the recipient's public key. Only the holder of the matching RSA private key can recover the AES key — and therefore decrypt the message.

The server receives an **encrypted_key** field that it cannot decrypt (no private key) and a **ciphertext** that is meaningless without the AES key. End-to-end confidentiality is mathematically guaranteed.

2.2 Why Passwords are Hashed and Not Encrypted

Encryption is reversible: anyone with the encryption key can recover the original password. A compromised server or malicious administrator could decrypt all stored passwords. Encryption also provides no protection against database breaches if the key is on the same system.

Password **hashing** is a one-way function — the original password cannot be recovered from the stored hash. When a user logs in, we hash their provided password with the same parameters and compare — we never need to recover the original.

We use **PBKDF2-HMAC-SHA256 with 600,000 iterations** (per OWASP 2023 recommendations) combined with a **random per-user salt**. The salt prevents rainbow-table attacks: two users with the same password produce completely different stored hashes. The high iteration count makes brute-force attacks computationally expensive.

2.3 Why Sessions are Needed

HTTP is a stateless protocol. Without sessions, the server cannot know who is making a request. The alternatives — sending the password with every request, or signing every request with the private key — both increase the attack surface and add complexity.

Sessions provide a time-bounded, revocable authentication token. After successful password verification, the server issues a **256-bit random session token** generated by `secrets.token_hex(32)`, which is cryptographically secure and unguessable. Sessions can be invalidated on logout, on timeout, or if a compromise is suspected.

2.4 Why Authorization is Enforced Server-Side

Authorization cannot be trusted to the client. A malicious client could simply remove the check, modify a request, or send a crafted token claiming to be a different user. The server must independently verify that the authenticated session user is permitted to perform each operation.

Our server enforces the following ACL on every message retrieval:

```
if message.recipient != session_user:  
    return 403 Forbidden
```

This means even an authenticated user cannot read another user's messages. The enforcement happens entirely on the server, after session validation, with no reliance on client-provided claims.

2.5 PKI Trust Model and Assumptions

Our system implements a **Trust-On-First-Use (TOFU)** PKI model with a server-side public key registry.

How it works: When a user registers, they generate a local RSA key pair and upload their public key to the server. When a sender wants to message a recipient, they query the server's registry for the recipient's current public key, then use it to wrap the AES key.

Trust assumptions:

- The server is trusted for key distribution but NOT for message confidentiality. If the server is compromised, an attacker could substitute a malicious public key and perform a Man-in-the-Middle attack.
- TLS protects key retrieval from network-level MitM. The client verifies the server's TLS certificate before trusting any response.
- Private keys never leave the client. The security of the entire system rests on the client's private key remaining secret and the passphrase remaining confidential.

3. Transport Layer Security (TLS)

3.1 Why HTTPS is Required

Without TLS, all communication travels in plaintext over the network. Any attacker with network access can:

- Read session tokens from the Authorization header → full account takeover
- Intercept public keys during registration → MitM the entire E2E encryption
- Observe communication metadata: who talks to whom, when, and how frequently

TLS provides three guarantees: **confidentiality** (data encrypted in transit), **integrity** (tampering is detectable), and **server authentication** (client verifies it is talking to the correct server via certificate).

3.2 Certificate Setup

We generate a self-signed X.509 certificate using the *cryptography* Python library (equivalent to OpenSSL). The certificate includes a SubjectAlternativeName extension for 127.0.0.1 and localhost. The client is configured with *verify=cert.pem*, so it explicitly trusts only this certificate and rejects any other — preventing MitM attacks using fraudulent certificates.

The server is configured to bind only to the HTTPS port (5443). Plain HTTP is never served.

4. End-to-End Encryption Data Flow

4.1 Sending a Message (Sender Side)

Step	Action	Code
1	Fetch recipient's RSA public key from server registry	GET /public_key/{recipient}
2	Generate a fresh 256-bit AES key (per-message)	aes_key = os.urandom(32)
3	Encrypt plaintext with AES-256-GCM	ciphertext, nonce, tag = encrypt_aes_gcm(msg, aes_key)
4	Encrypt AES key with recipient's RSA public key	enc_key = rsa_encrypt(aes_key, recipient_pubkey)
5	POST encrypted bundle to server	POST /send {enc_key, ciphertext, nonce, tag}

4.2 Receiving a Message (Receiver Side)

Step	Action
1	Fetch encrypted messages from server (ACL enforced — only own messages returned)
2	Decode base64 fields: encrypted_key, ciphertext, nonce, tag

3	Decrypt AES key using own RSA private key (RSA-OAEP)
4	Decrypt ciphertext using AES-256-GCM — also verifies integrity via auth tag
5	Display plaintext to user

4.3 What the Server Sees vs. What it Cannot Decrypt

Field	Server Stores It?	Server Can Decrypt?	Why
ciphertext	Yes	NO	Needs AES key — never sent to server
encrypted_key	Yes	NO	Needs recipient's private key — client-only
nonce	Yes	N/A	Not secret; needed by receiver only
tag	Yes	N/A	Integrity check; receiver verifies
Plaintext message	NEVER	N/A	Never transmitted to server
AES key	NEVER	N/A	Generated and discarded on client

5. Remaining Attack Vectors

No security system is perfect. The following attacks remain possible and would require additional mitigations beyond the scope of this assignment:

Server Compromise / Key Substitution

If an attacker gains control of the server, they can replace stored public keys, causing future senders to encrypt to the attacker's key instead. This is inherent to centralized key distribution. Mitigation: a transparency log or out-of-band key fingerprint verification.

Endpoint Compromise

If a client device is compromised (malware, physical access), the attacker can steal the private key file or read plaintext from memory. Mitigations: full-disk encryption, hardware security modules (HSMs/TPMs), and short key lifetimes.

Session Token Theft

Session tokens are stored in memory and transmitted over TLS. If TLS is broken or a user's machine is compromised, tokens can be stolen. Short-lived tokens with automatic expiry reduce the window of exposure.

Denial of Service

The server has no rate limiting. An attacker could flood /register or /login to exhaust resources. Mitigation: rate limiting, CAPTCHA, and IP-based blocking.

No Forward Secrecy on Stored Messages

If a recipient's private key is later compromised, all past messages encrypted to that key become readable. Forward secrecy (e.g., Signal's Double Ratchet protocol) generates ephemeral keys per session so past messages remain protected even if a long-term key is compromised.

Metadata Leakage

The server knows who communicates with whom, when, and how frequently — even though it cannot read content. This traffic analysis can be valuable to an adversary. Mitigation: mixnets or onion routing (e.g., Tor).

Offline Dictionary Attacks on Password Hashes

If the database is stolen, offline brute-force attacks against PBKDF2 hashes are possible, albeit slow. Argon2id would provide better resistance against GPU-accelerated cracking due to its memory-hardness property.

6. Summary of Security Controls Implemented

Component	Control	Implementation Detail
Identity	Registration with public key upload	POST /register with PEM-encoded RSA public key
Authentication	Password hashing	PBKDF2-HMAC-SHA256, 600K iterations, random 32-byte salt per user
Sessions	Secure token generation	secrets.token_hex(32) — 256-bit cryptographically secure random token
Authorization	Server-side ACL enforcement	Recipient check on every fetch; 403 returned on violation
Message Confidentiality	End-to-end encryption	AES-256-GCM for ciphertext; RSA-OAEP-SHA256 for key wrapping
Key Exchange	PKI public key registry	Server stores public keys only; private keys never leave client
Transport Security	TLS / HTTPS	Self-signed X.509 cert; server binds HTTPS only; client verifies cert
Message Integrity	Authenticated encryption	AES-GCM 128-bit authentication tag verified on every decryption
Privacy	Server-side blindness	Server stores only ciphertext — no plaintext ever transmitted to server